

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA0624	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	Total Marks:	100
CO2 Statement:	<i>Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)</i>		
Student Name:	Karthikeyan k	Reg. No.:	192425072
Section / Batch:	B.Tech (AIML) 3 rd Year	Date:	25/07/26

Instructions to Students

- Answer ALL debugging and optimisation tasks. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions	Marks
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1 - Q2	100 (2 × 50)
GRAND TOTAL:		100 Marks	

SECTION A – DEBUGGING & OPTIMISATION TASK

Code of Conduct

I swetha s(192424049) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: swetha S _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, wellstructured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Question No.	Problem Identification (20%)	Debugging (25%)	Optimization (20%)	Analysis (20%)	Code Quality (15%)	Total Marks (Each – 50)
--------------	------------------------------	-----------------	--------------------	----------------	--------------------	-------------------------

Q1						
Q2						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____ Date: _____	Signature: _____ Date: _____	Signature: _____ Date: _____

Q5. Debugging Binary Search Code

The following Binary Search implementation fails for certain inputs due to incorrect boundary updates. Identify all logical errors and explain their causes. Debug the code to handle all edge cases correctly. Optimize the implementation to avoid overflow in mid calculation. Analyze its performance in best, average, and worst cases. Rewrite the corrected code with proper documentation.

```
def binary_search(arr, key):
    low, high = 0, len(arr)-1
    while low < high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            high = mid - 1 # ERROR
        else:
            low = mid + 1 # ERROR
    return -1
```

Errors in the Code

- **Loop Condition Error:** while low < high does not check the last remaining element.
- **Boundary Update Error:** high = mid - 1 is used when arr[mid] < key, which searches the wrong half.
- **Boundary Update Error:** low = mid + 1 is used when arr[mid] > key, which also searches the wrong half.
- **Overflow Issue:** mid = (low + high) // 2 may cause overflow in some programming languages.

Root Causes

- The loop should continue while low <= high.

- If $\text{arr}[\text{mid}] < \text{key}$, search the **right half** by updating $\text{low} = \text{mid} + 1$.
- If $\text{arr}[\text{mid}] > \text{key}$, search the **left half** by updating $\text{high} = \text{mid} - 1$.
- Use $\text{mid} = \text{low} + (\text{high} - \text{low}) // 2$ to avoid overflow.

Corrected Steps

- Initialize $\text{low} = 0$ and $\text{high} = \text{len}(\text{arr}) - 1$.
- Continue searching while $\text{low} \leq \text{high}$.
- Calculate mid using the overflow-safe formula.
- Compare the middle element with the key.
- Return the index if found; otherwise return -1.

Time Complexity

- **Best Case:** $O(1)$
- **Average Case:** $O(\log n)$
- **Worst Case:** $O(\log n)$
- **Space Complexity:** $O(1)$

Corrected Code

```
def binary_search(arr, key):
    low, high = 0, len(arr) - 1

    while low <= high:
        # Overflow-safe middle calculation
        mid = low + (high - low) // 2

        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1

    return -1
```

Conclusion

The corrected Binary Search searches the correct half of the array, checks all elements, avoids overflow in middle calculation, and achieves **$O(\log n)$** time complexity.

Q6. Debugging Bubble Sort Code

The following Bubble Sort implementation does not correctly sort the array and performs unnecessary iterations. Identify logical errors in loop conditions and swapping mechanism, and explain their causes. Debug the code step-by-

step to ensure correct sorting. Optimize the algorithm by reducing unnecessary passes using a flag. Analyze the time complexity of the corrected version.

Rewrite the final optimized code with proper comments. def

```
bubble_sort(arr):  
    n = len(arr)  
    for i in range(n):  
        for j in range(n-1):  
            if arr[j] < arr[j+1]: # ERROR  
                arr[j], arr[j+1] = arr[j+1], arr[j]  
        return arr
```

Errors in the Code

- **Comparison Error:** `arr[j] < arr[j+1]` sorts the array in descending order instead of ascending order.
- **Loop Condition Error:** Inner loop always runs $n-1$ times, causing unnecessary comparisons.
- **Optimization Missing:** No flag is used to stop when the array is already sorted.

Root Causes

- The comparison operator should be `>` for ascending order.
- The inner loop should reduce its range after every pass.
- A flag should stop the algorithm if no swaps occur.

Corrected Steps

- Compare adjacent elements.
- Swap if the left element is greater.
- Reduce the inner loop range after each pass.
- Stop early if no swaps occur.

Time Complexity

- **Best Case:** $O(n)$
- **Average Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$
- **Space Complexity:** $O(1)$

Corrected Code

```
def bubble_sort(arr):  
    n = len(arr)
```

```

    for i in range(n):
        swapped = False

        for j in range(0, n - i - 1):
            if
arr[j] > arr[j + 1]:
                arr[j], arr[j +
1] = arr[j + 1], arr[j]
            swapped =
True

        if not swapped:
            break

    return arr

```

Conclusion

The corrected Bubble Sort correctly sorts the array in ascending order. Using a swap flag reduces unnecessary passes, improving the best-case performance to **$O(n)$** .

Q7. Debugging Insertion Sort Code

The following Insertion Sort code fails to correctly place elements in sorted order. Identify errors in shifting logic and loop conditions. Explain why incorrect insertion occurs. Debug the algorithm step-by-step to ensure correct functionality. Optimize the code for better readability. Analyze the time complexity. Rewrite the corrected version with proper structure and comments.

```

def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j > 0 and arr[j] > key: # ERROR
            arr[j+1] = arr[j]
            j -= 1
        arr[j] = key # ERROR
    return arr

```

Errors in the Code

- **Loop Condition Error:** while $j > 0$ skips comparison with the first element.
- **Insertion Error:** $\text{arr}[j] = \text{key}$ places the key in the wrong position.

Root Causes

- The condition should be $j \geq 0$.
- After shifting elements, insert the key at $\text{arr}[j + 1]$.

Corrected Steps

- Store the current element as key.
- Shift all larger elements one position to the right.
- Insert the key into its correct position.

Time Complexity

- **Best Case:** $O(n)$
- **Average Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$
- **Space Complexity:** $O(1)$

Corrected Code

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        arr[j + 1] = key

    return arr
```

Conclusion

The corrected Insertion Sort properly shifts elements and inserts each element into its correct position, producing a correctly sorted array.

Q8. Debugging String Matching Code

The given brute-force string matching code fails to detect all pattern occurrences due to incorrect loop bounds. Identify logical errors in indexing and comparison. Explain why some matches are skipped. Debug the code to correctly find all occurrences. Optimize the implementation to avoid redundant comparisons. Analyze the time complexity of the corrected version. Rewrite the final code with proper comments.

```
def string_match(text, pattern):
    n, m = len(text), len(pattern)
    for i in range(n - m):
        for j in range(m):
            if text[i+j] != pattern[j]:
                break
        if j == m-1:
```

```
print("Match at index", i)
```

Errors in the Code

- **Loop Bound Error:** `range(n - m)` skips the last possible matching position.
- **Comparison Error:** Some valid matches are missed because the last index is not checked.

Root Causes

- The outer loop should run until `range(n - m + 1)`.
- Every possible starting position must be checked.

Corrected Steps

- Loop from index 0 to `n-m`.
- Compare each character of the pattern.
- Print the index if the entire pattern matches.

Time Complexity

- **Best Case:** $O(n)$
- **Average Case:** $O(nm)$
- **Worst Case:** $O(nm)$
- **Space Complexity:** $O(1)$

Corrected Code

```
def string_match(text, pattern):  
    n, m = len(text), len(pattern)  
  
    for i in range(n - m + 1):  
        for j in range(m):  
            if text[i + j] != pattern[j]:  
                break  
        else:  
            print("Match at index", i)
```

Conclusion

The corrected String Matching algorithm checks every possible starting position and successfully finds all pattern occurrences.

Q9. Debugging Maximum Element Search Code

The following code attempts to find the maximum element in an array but returns incorrect results for some inputs. Identify errors in initialization and comparison logic. Explain why incorrect values are produced. Debug the

algorithm step-by-step to fix the issue. Optimize the implementation for clarity and correctness. Analyze the time complexity. Provide a clean and corrected version of the code.

```
def find_max(arr):
    max_val = 0 # ERROR for
    for i in range(len(arr)):
        if arr[i] < max_val: # ERROR
            max_val = arr[i]
    return max_val
```

Errors in the Code

- **Initialization Error:** `max_val = 0` fails for arrays containing only negative numbers.
- **Comparison Error:** `arr[i] < max_val` finds the minimum instead of the maximum.

Root Causes

- Initialize `max_val` with the first array element.
- Use the `>` operator to find the largest value.

Corrected Steps

- Set `max_val = arr[0]`.
- Compare each element with the current maximum.
- Update the maximum when a larger element is found.

Time Complexity

- **Best Case:** $O(n)$
- **Average Case:** $O(n)$
- **Worst Case:** $O(n)$
- **Space Complexity:** $O(1)$

Corrected Code

```
def find_max(arr):
    max_val = arr[0]

    for i in range(1, len(arr)):
        if arr[i] > max_val:
            max_val = arr[i]

    return max_val
```

Conclusion

The corrected algorithm correctly finds the maximum element, including in arrays containing negative numbers.

Q10. Debugging Pair Sum Code (Brute Force)

The following brute force code is intended to find a pair of elements whose sum equals a target value, but it fails in certain cases. Identify logical errors in loop structure and return conditions. Explain why valid pairs may not be detected. Debug the code step-by-step to ensure correctness. Optimize the implementation if possible. Analyze the time complexity of the corrected solution. Rewrite the final version with proper documentation.

```
def pair_sum(arr, target):  
    for i in range(len(arr)):  
        for j in range(len(arr)):  
            if i != j and arr[i] + arr[j] == target:  
                return (i, j)  
    return None
```

Errors in the Code

- **Loop Structure Error:** The inner loop starts from 0, causing duplicate pair checks.
- **Duplicate Comparison Error:** The same pair is checked twice (e.g., (i, j) and (j, i)).
- **Unnecessary Self-Check:** $i \neq j$ is required because the loop includes the same index.

Root Causes

- The inner loop should start from $i + 1$.
- Each pair should be checked only once.
- Removing duplicate checks improves efficiency.

Corrected Steps

- Use the outer loop for the first element.
- Use the inner loop starting from $i + 1$.
- Return the pair when the target sum is found.
- Return None if no pair exists.

Time Complexity

- **Best Case:** $O(1)$
- **Average Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$
- **Space Complexity:** $O(1)$

Corrected Code

```
def pair_sum(arr, target):  
    for i in range(len(arr)):  
        for j in range(i + 1, len(arr)):  
            if arr[i] + arr[j] == target:  
                return (i, j)  
  
    return None
```

Conclusion

The corrected Pair Sum algorithm checks each unique pair exactly once, avoids duplicate comparisons, and correctly finds a pair whose sum equals the target while maintaining **$O(n^2)$** time complexity.